Operating Systems

Chapter 3 Memory Management

Tien Pham Van, Dr. rer. nat.

(*Lecture compiled with reference to other presentations*)

Concept

- Memory management is the act of managing computer memory.
- It involves providing ways to allocate portions of memory to programs at their request, and freeing it for reuse when no longer needed.
- The management of main memory is critical to the computer system, particular embedded systems





Memory Management Requirements

- Relocation
  - Programmer does not know where the program will be placed in memory when it is executed
  - While the program is executing, it may be swapped to disk and returned to main memory at a different location (relocated)
  - Memory references must be translated in the code to actual physical memory address

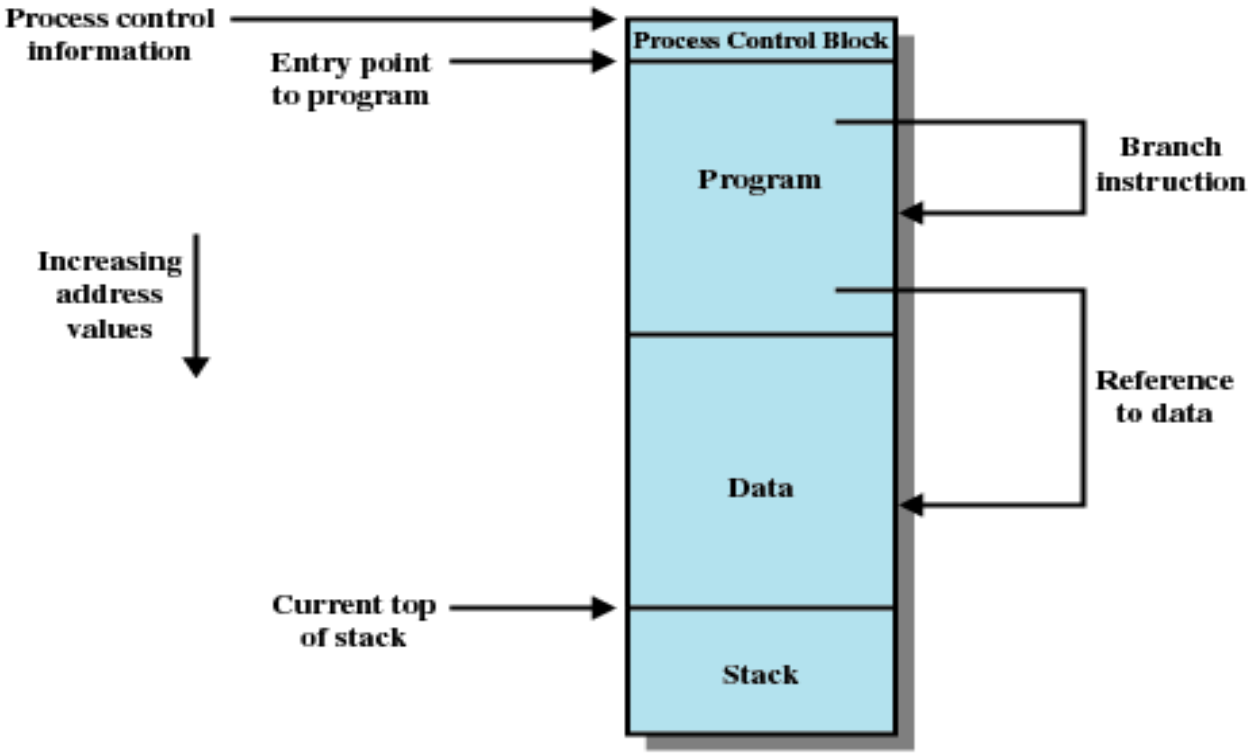


Figure 7.1 Addressing Requirements for a Process

Memory Management Requirements

- Protection
  - Processes should not be able to reference memory locations in another process without permission
  - Impossible to check absolute addresses at compile time
  - Must be checked at run time
  - Memory protection requirement must be satisfied by the processor (hardware) rather than the operating system (software)
    - Operating system cannot anticipate all of the memory references a program will make

Memory Management Requirements

- Sharing
  - Allow several processes to access the same portion of memory
  - Better to allow each process access to the same copy of the program rather than have their own separate copy

Memory Management Requirements

- Logical Organization
  - Programs are written in modules
  - Modules can be written and compiled independently
  - Different degrees of protection given to modules (read-only, execute-only)
  - Share modules among processes

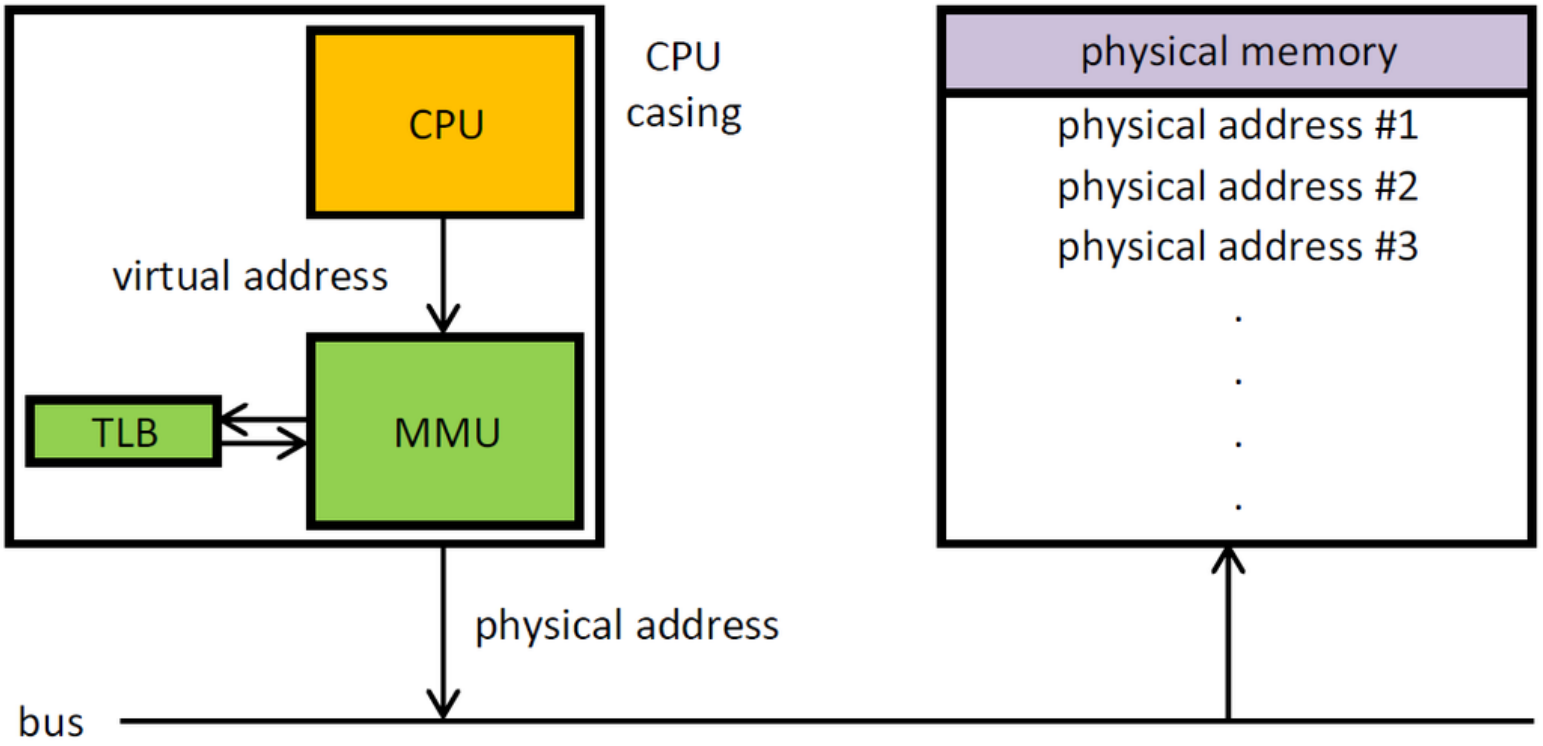
- Operating Systems 2 - Memory Manager - YouTube

https://www.youtube.com/watch?v=qdkxXygc3rE&t=195s

Memory Management Requirements

- Physical Organization
  - Memory available for a program plus its data may be insufficient
    - Overlaying allows various modules to be assigned the same region of memory
  - Programmer does not know how much space will be available

MMU



CPU: Central Processing Unit

MMU: Memory Management Unit

TLB: Translation lookaside buffer

Fixed Partitioning

- Equal-size partitions
  - Because all partitions are of equal size, it does not matter which partition is used
  - Main memory use is inefficient. Any program, no matter how small, occupies an entire partition. This is called internal fragmentation.
- Unequal-size partitions
  - Can assign each process to the smallest partition within which it will fit
  - Queue for each partition
  - Processes are assigned in such a way as to minimize wasted memory within a partition

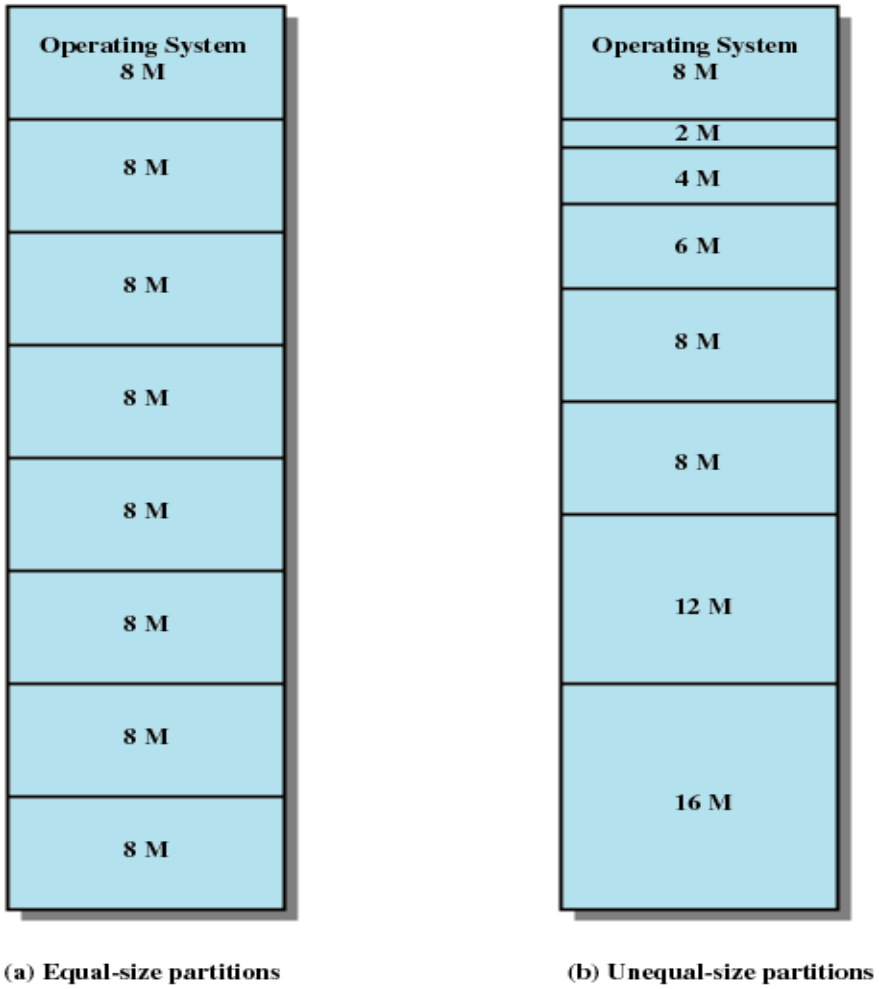


Figure 7.2 Example of Fixed Partitioning of a 64-Mbyte Memory

Dynamic Partitioning

- Partitions are of variable length and number
- Process is allocated exactly as much memory as required
- Eventually get holes in the memory. This is called external fragmentation
- Must use compaction to shift processes so they are contiguous and all free memory is in one block

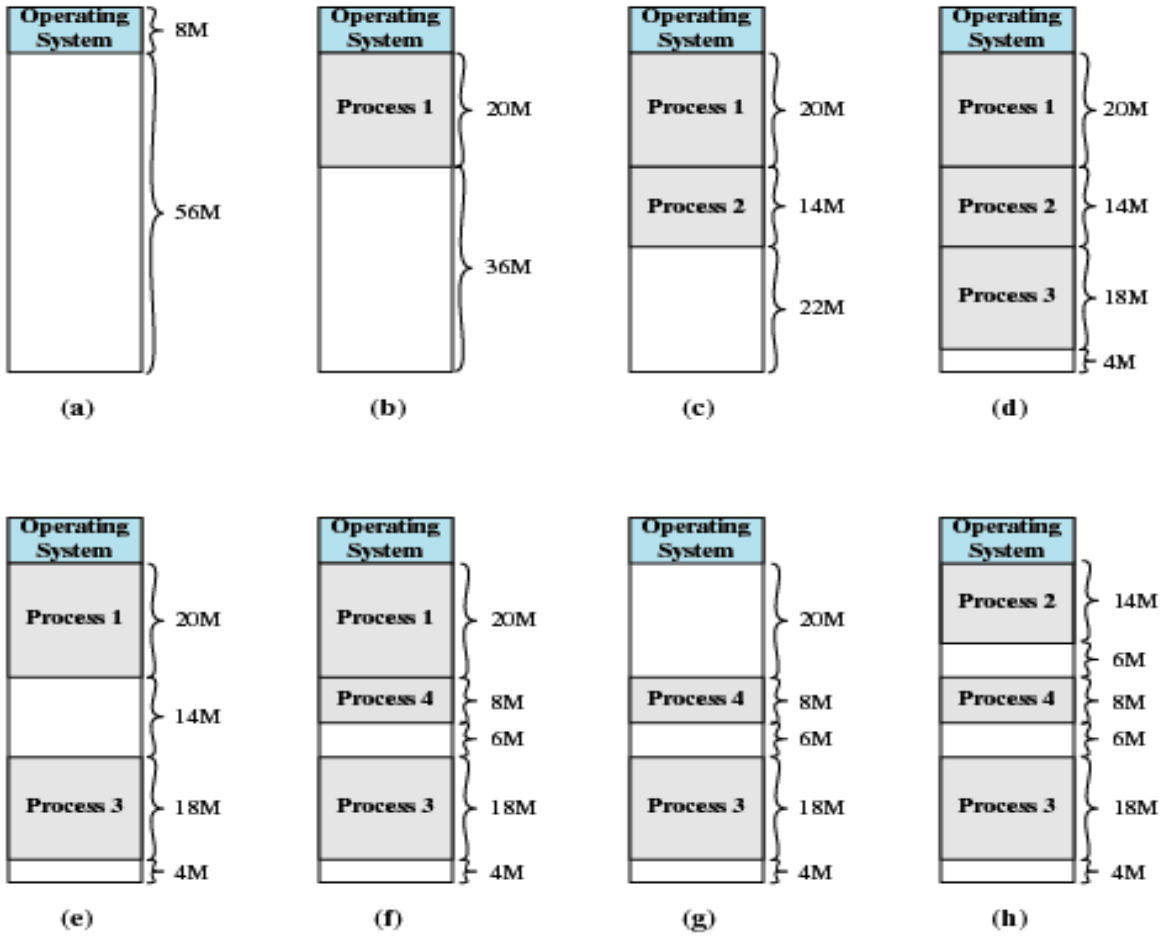


Figure 7.4 The Effect of Dynamic Partitioning

Dynamic Partitioning Placement Algorithm

- Operating system must decide which free block to allocate to a process
- Best-fit algorithm
  - Chooses the block that is closest in size to the request
  - Worst performer overall
  - Since smallest block is found for process, the smallest amount of fragmentation is left
  - Memory compaction must be done more often

Dynamic Partitioning Placement Algorithm

- First-fit algorithm
  - Scans memory form the beginning and chooses the first available block that is large enough
  - Fastest
  - May have many process loaded in the front end of memory that must be searched over when trying to find a free block

Dynamic Partitioning Placement Algorithm

- Next-fit
  - Scans memory from the location of the last placement
  - More often allocate a block of memory at the end of memory where the largest block is found
  - The largest block of memory is broken up into smaller blocks
  - Compaction is required to obtain a large block at the end of memory

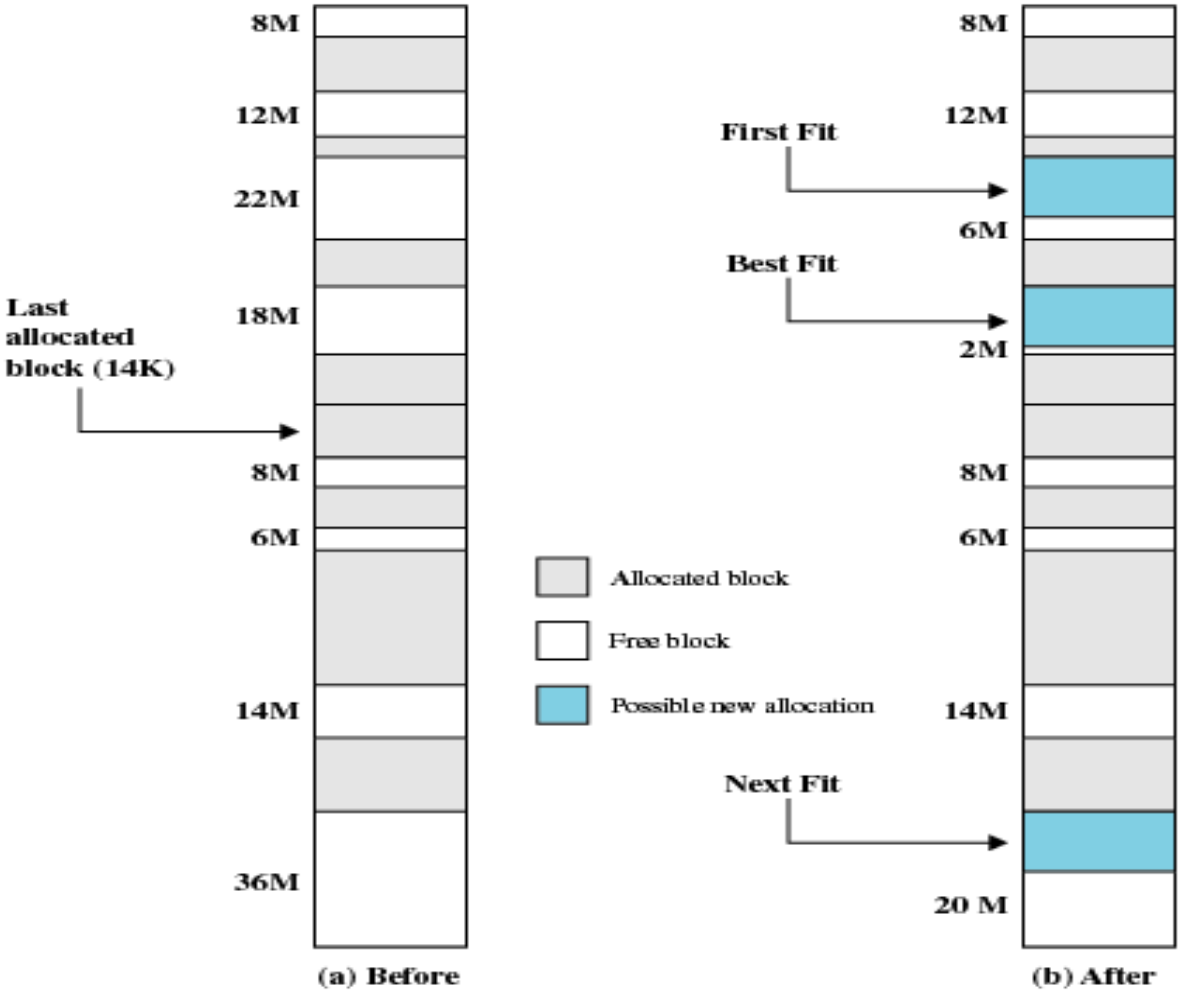


Figure 7.5 Example Memory Configuration Before and After Allocation of 16 Mbyte Block

Buddy System

- Entire space available is treated as a single block of 2^U
- If a request of size s such that $2^{U-1} < s <= 2^U$, entire block is allocated
  - Otherwise block is split into two equal buddies
  - Process continues until smallest block greater than or equal to s is generated

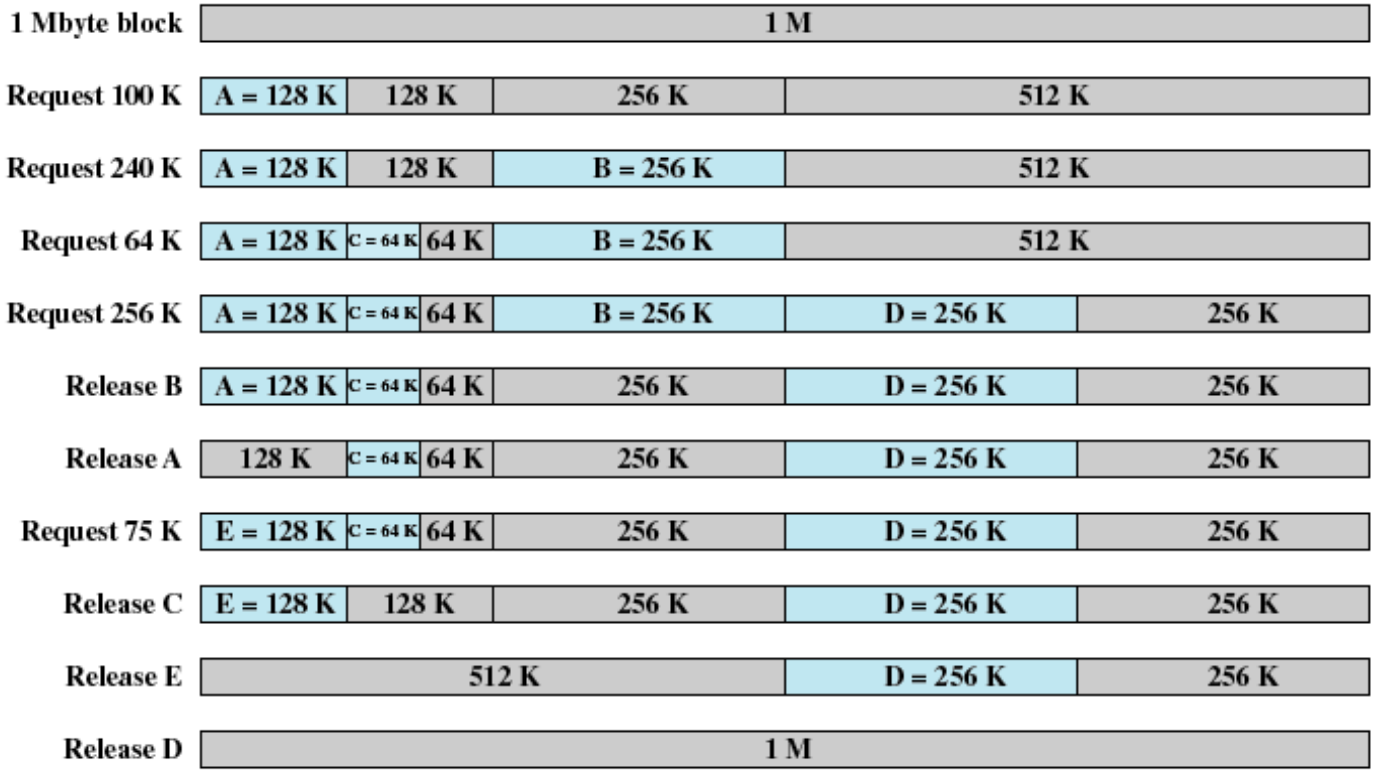


Figure 7.6 Example of Buddy System

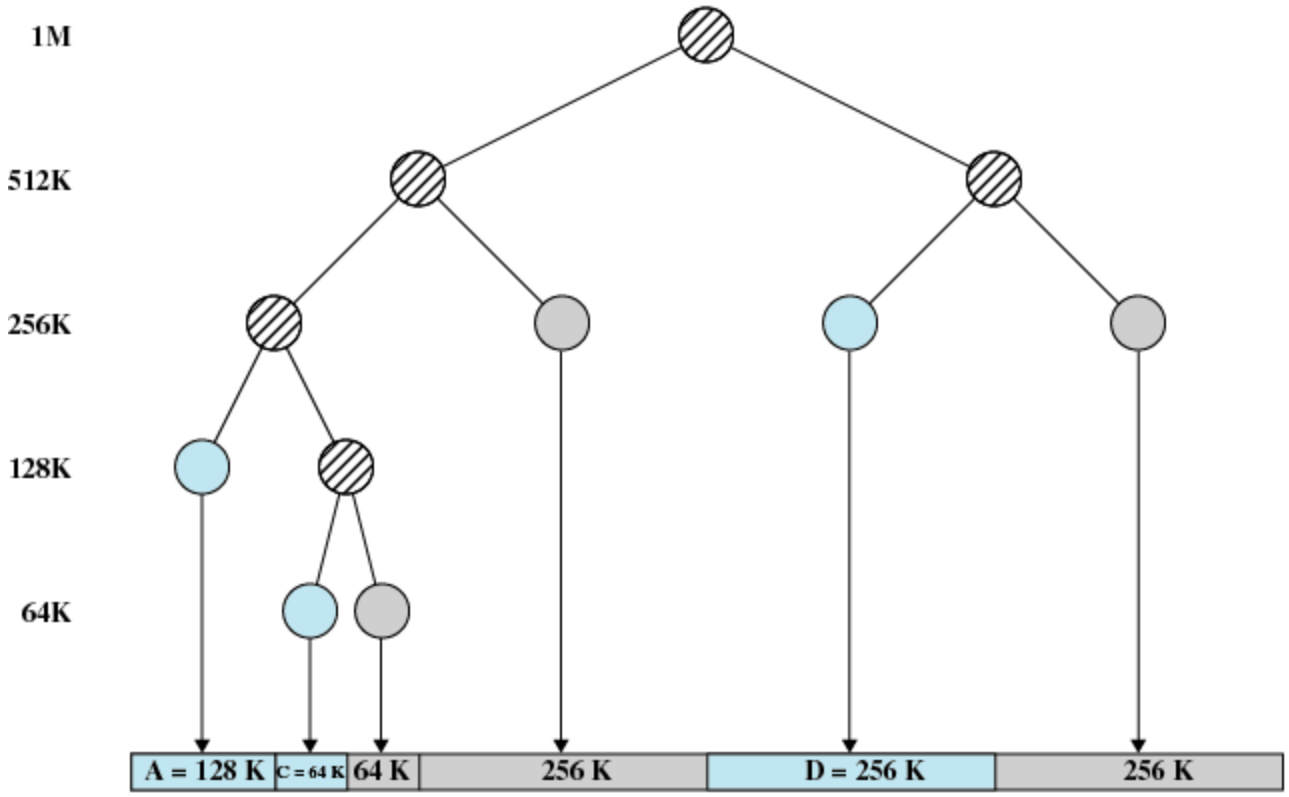


Figure 7.7 Tree Representation of Buddy System

Addresses

- Logical
  - Reference to a memory location independent of the current assignment of data to memory
  - Translation must be made to the physical address
- Relative
  - Address expressed as a location relative to some known point
- Physical
  - The absolute address or actual location in main memory

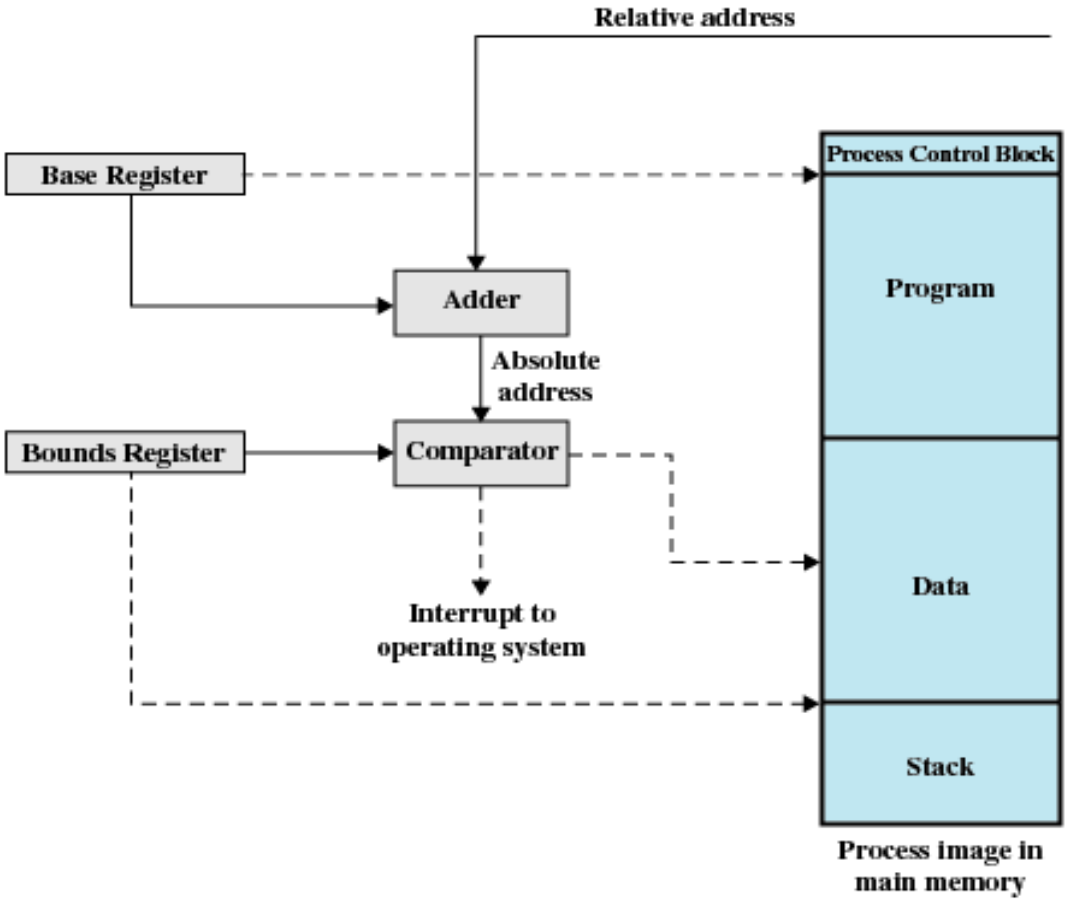


Figure 7.8 Hardware Support for Relocation

Paging

- Partition memory into small equal fixed-size chunks and divide each process into the same size chunks
- The chunks of a process are called pages and chunks of memory are called frames
- Operating system maintains a page table for each process
  - Contains the frame location for each page in the process
  - Memory address consist of a page number and offset within the page

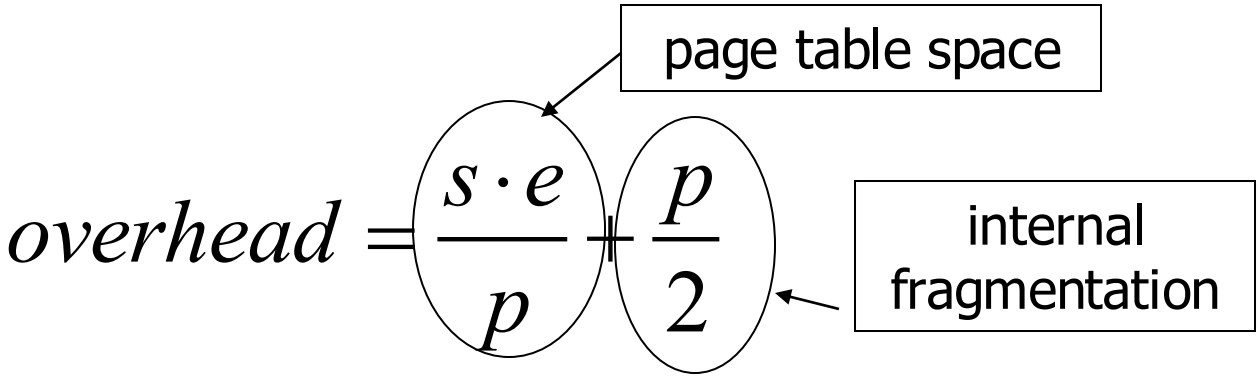
Page Size (1)

Small page size

- Advantages
  - less internal fragmentation
  - better fit for various data structures, code sections
  - less unused program in memory
- Disadvantages
  - programs need many pages, larger page tables

Page Size (2)

- Overhead due to page table and internal fragmentation



- Where
  - s = average process size in bytes
  - p = page size in bytes
  - e = page entry

Optimized when

$$p = \sqrt{2se}$$

Page size (3)

Architecture	Smallest page size	Larger page sizes
x86-64[18]	4 KiB	2 MiB, 1 GiB (only when the CPU has PDPE1GB flag)
Power ISA[21]	4 KiB	64 KiB, 16 MiB, 16 GiB
SPARC v8 with SPARC Reference MMU[22]	4 KiB	256 KiB, 16 MiB
UltraSPARC Architecture 2007[23]	8 KiB	64 KiB, 512 KiB (optional), 4 MiB, 32 MiB (optional), 256 MiB (optional), 2 GiB (optional), 16 GiB (optional)
ARMv7[24]	4 KiB	64 KiB, 1 MiB ("section"), 16 MiB ("supersection") (defined by a particular implementation)

Page size (4)

```
#include <stdio.h>
#include <unistd.h> /* sysconf(3) */

int main(void)
{
        printf("The page size for this system is %ld bytes.\n",
                sysconf(_SC_PAGESIZE)); /* _SC_PAGE_SIZE is OK too. */

        return 0;
}
```

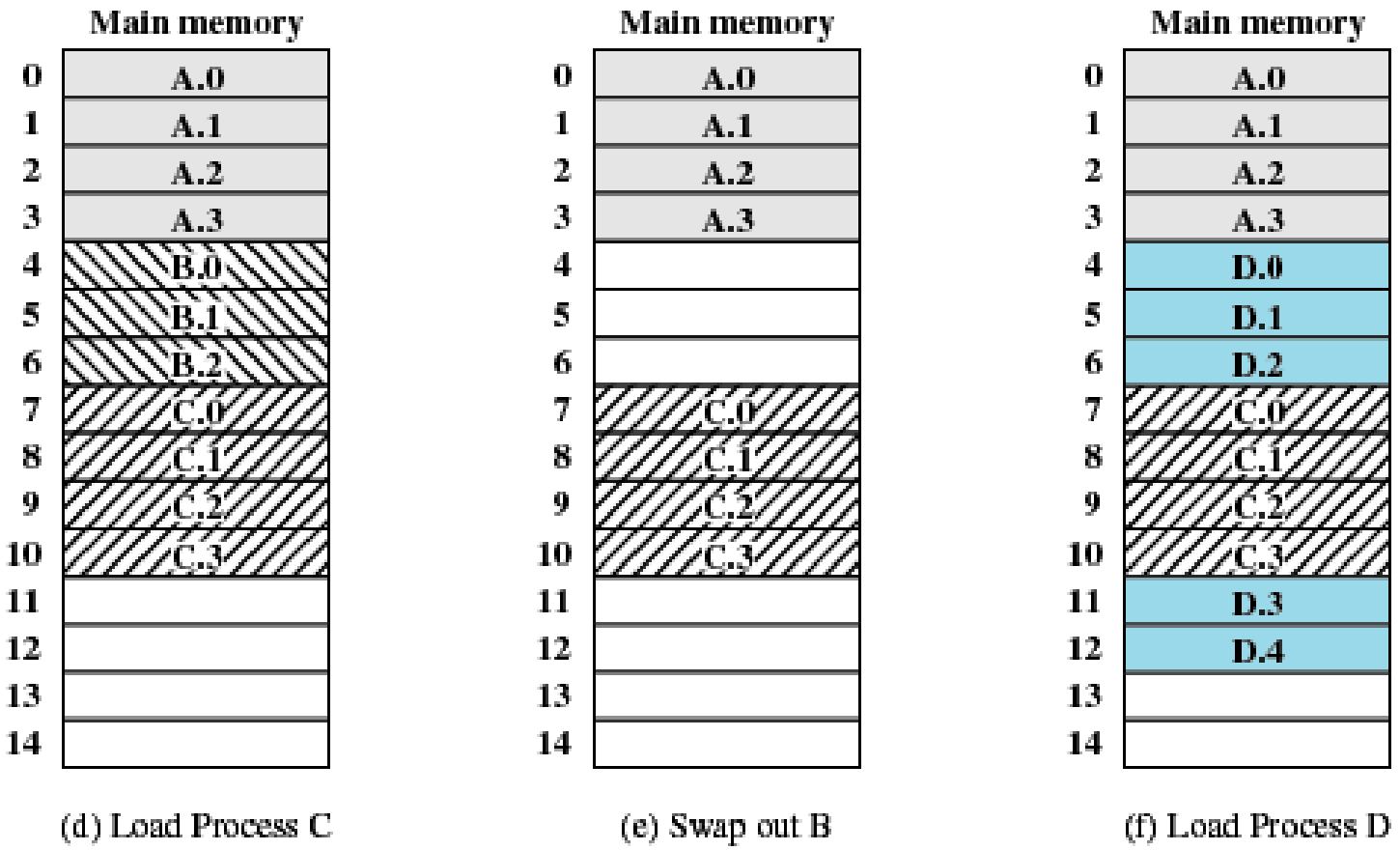
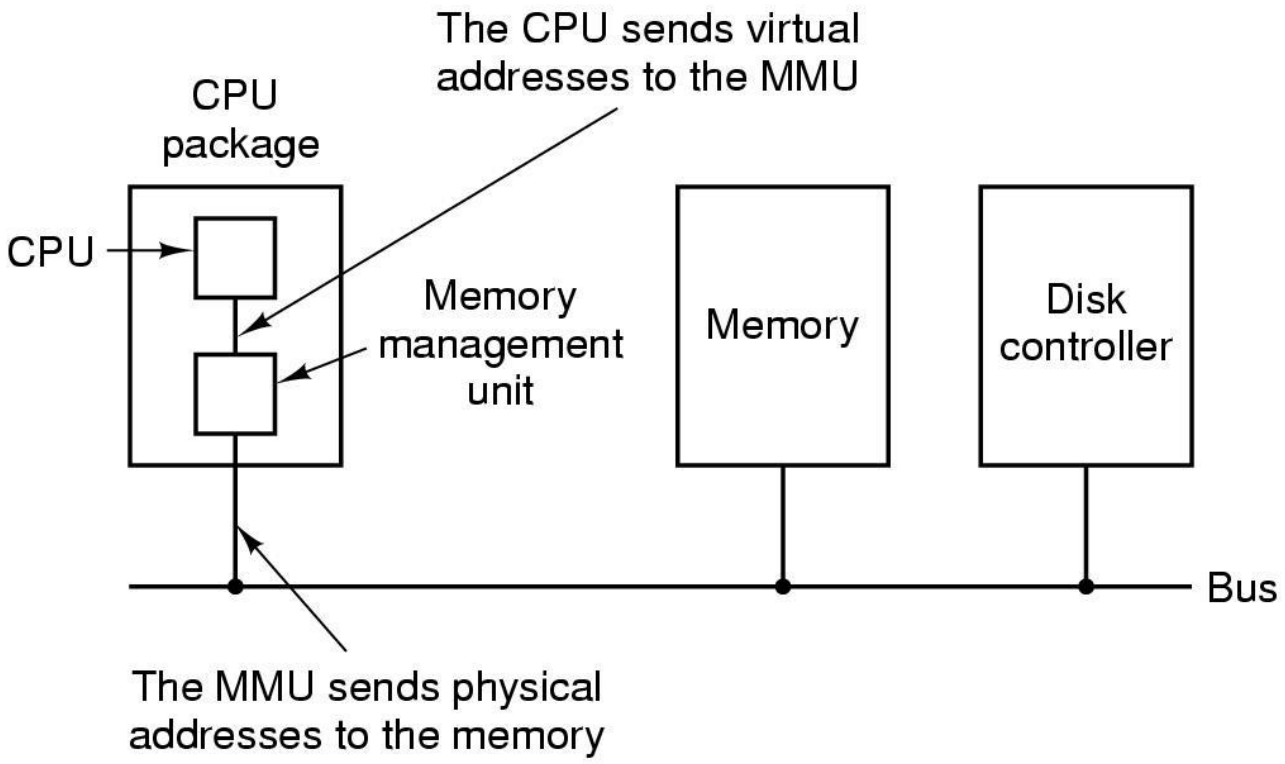


Figure 7.9 Assignment of Process Pages to Free Frames

- https://www.youtube.com/watch?v=AKGtJAi4wGo&ab_channel=Xoviabcs

https://www.youtube.com/watch?v=AKGtJAi4wGo

Virtual Memory Paging



The position and function of the MMU

Paging and translation

The relation between virtual addresses and physical memory addresses given by page table

